OOP with Java 1

Re-exam project

Group 03: Hansmar Poulsen
Student id: 2004.024

August 2026

Contents

1 Introduction

This project extends the roguelike game from the previous project with a system for status effects. A status effect is a temporary effect that changes the state or behavior of an entity for a fixed number of turns. The extension adds four effects, one for each behavior required by the assignment: a damage-over-time effect, a heal-over-time effect, an effect that prevents movement, and an effect that alters the damage an entity deals.

I approached this extension the same way as the previous project. I built one effect end to end first so that the base class, the per-turn update, and the messaging were settled before adding the rest. Once that worked, the rest was a new subclass and a source (tile or item) that applies it.

2 Description of the solution

2.1 The `StatusEffect` base class

All status effects extend the abstract class `StatusEffect`. It holds the state common to every effect, a name and a counter of remaining turns and the code that every effect shares: decrementing the counter each turn and reporting when the effect has expired. Each concrete effect supplies only `applyEffect`, the per-turn behaviour.

`StatusEffect` is an abstract class rather than an interface because it carries state: the remaining-turns counter. An interface cannot hold that.

The base class is shown below, abbreviated, from `StatusEffect.java`:

```
public abstract class StatusEffect {
    private final String name;
    private int remainingTurns;

    protected StatusEffect(String name, int durationTurns) {
        this.name = name;
        this.remainingTurns = durationTurns;
    }

    public boolean isExpired() { return remainingTurns <= 0; }

    public final String onTurn(LivingEntity target) {
        remainingTurns--;
        return applyEffect(target);
    }

    protected abstract String applyEffect(LivingEntity target);
}
```

2.2 The capability interfaces

Two behaviours do not belong on the base class, because they are not shared by all effects and carry no state of their own: blocking movement, and altering damage. These are expressed as interfaces: `MovementBlocking` and `DamageModifying` that individual effects implement. `FreezeEffect` implements `MovementBlocking` and `StrengthEffect` implements `DamageModifying`.

The engine never asks which concrete effect it is dealing with. `LivingEntity.isMovementBlocked()` checks whether any active effect is a `MovementBlocking`, and `modifyOutgoingDamage()` passes

the base damage through any `DamageModifying` effect. Both check against the interface, not the concrete class. The two methods below are from `LivingEntity.java`.

```
public boolean isMovementBlocked() {
    for (StatusEffect e : effects) {
        if (e instanceof MovementBlocking) return true;
    }
    return false;
}

public int modifyOutgoingDamage(int base) {
    int dmg = base;
    for (StatusEffect e : effects) {
        if (e instanceof DamageModifying dm) dmg = dm.modifyDamage(dmg);
    }
    return Math.max(dmg, 0);
}
```

The division is deliberate: the abstract class holds the state shared by every effect, and the interfaces express capabilities that cut across the effect types. Each tool is used where it fits.

2.3 The four effects

`BurningEffect` deals a fixed amount of damage each turn and is applied when the player walks into a fire tile. `RegenerationEffect` heals a fixed amount each turn and is applied by a water tile. `FreezeEffect` prevents movement and is applied by a freeze trap. `StrengthEffect` increases the damage the player deals and is applied by a strength potion.

2.4 Integration into the entity and the turn

Each `LivingEntity` holds its active effects in a private list, reachable only through `applyStatusEffect` and `tickStatusEffects`. Because effects live on `LivingEntity`, both the player and the enemies can carry them.

Effects are updated once per turn. Enemies are updated in `Dungeon.tickAll`; the player is updated separately in `Game.tick`, because the player is not stored in the dungeon's entity list. Freeze is enforced in the player's movement branch: a frozen player's move is skipped, but the turn still passes, so the effect counts down and eventually expires. Strength is applied in combat, where the player's attack is passed through `modifyOutgoingDamage` before it is dealt.

An effect applies on each turn of its duration, including the turn it expires, and is then removed. Because every effect shares the same per-turn update in `tickStatusEffects`, this timing is consistent across all four effects: an effect declared for n turns acts on n turns, and the entity is free of it on the following turn.

2.5 Duplicate handling

An entity can carry several effects at once. When the same effect is applied while it is already active, the duration is refreshed rather than stacked: `applyStatusEffect` checks whether an effect of the same class is present and, if so, resets its remaining turns instead of adding a second copy. Refreshing was chosen because it matches the intuition that standing in fire keeps you burning, and it avoids effects stacking without limit.

2.6 Messaging

The assignment requires that application, each turn, and expiration are communicated. Each of these produces a plain string: `onApply` when the effect is added, `applyEffect` each turn, and

`onExpire` when it ends. These strings are collected by `tickStatusEffects` and passed into the game's existing status-message line, so no new messaging mechanism was needed.

3 Testing

The effects are covered by a JUnit test suite written in the black-box style: each test constructs a `LivingEntity`, applies an effect, advances the turns by calling `tickStatusEffects` directly, and checks the result through public methods. The suite tests each effect (damage, healing and the health cap, movement blocking, damage alteration), the duplicate rule (that a repeated effect refreshes rather than stacks), the expiry message, and two effects active at the same time.

The tests do not run the game loop; they exercise the effect logic in isolation. This is possible because that logic is separated onto `LivingEntity` and the effect classes, rather than embedded in the game engine.

4 Shortcomings

- **Effects are only applied to the player.** The mechanism is general because effects live on `LivingEntity`, any enemy could carry one but every source I added targets the player. No source applies an effect to an enemy, so in the game enemies never experience status effects.
- **The tests exercise effects on the player only.** Because effects live on `LivingEntity`, an enemy should behave identically, but it is not applied to the game.

5 Reflection

The status effects themselves were straight forward to apply once the first one worked. Burning came first, and after that each effect was the same pattern with a different body.

The integration took more time than the effects. The player and the enemies are updated in different places because the player is not in the dungeon's entity list, and freeze had to skip the move while still letting the turn pass, which took a few tries to get right.

The design question of when to use an abstract class and when an interface came down to whether there was a shared state.